第5章 第6节 向量数据库（二）-进阶算法与Chroma实战

- 来源: https://class.imooc.com/lesson/5272#mid=104918
- 阶段: 阶段二: 【实战入门】大模型开发与知识接入
- 周: 第5周 RAG：知识接入技术
- 模块: 向量数据库
- 课节: 图文 1-2 向量数据库（二）-进阶算法与Chroma实战

1-2 向量数据库（二）-进阶算法与Chroma实战

第5章 第6节 向量数据库（二）-进阶算法与Chroma实战

6.1 LSH（局部敏感哈希）

核心思想：用哈希函数把相似向量映射到同一个桶

哈希函数

Hash的本质是将任意长度的输入转化为固定长度的编码。相同的输入一定产生相同的编码，不同的输入则产生不同的编码。

举个具体的例子

```
# 传统哈希（如MD5、SHA）的特点
import hashlib

text1 = "hello"
text2 = "hello"  # 完全相同
text3 = "hallo"  # 只有一个字母不同

hash1 = hashlib.md5(text1.encode()).hexdigest()
hash2 = hashlib.md5(text2.encode()).hexdigest()
hash3 = hashlib.md5(text3.encode()).hexdigest()

print(f"text1哈希: {hash1[:16]}...")  # 5d41402abc4b2a76...
print(f"text2哈希: {hash2[:16]}...")  # 5d41402abc4b2a76... (与text1相同)
print(f"text3哈希: {hash3[:16]}...")  # 437b930db84b8079... (完全不同)

1
2
3
4
5
6
7
8
9
10
11
12
13
14

预览
```

传统哈希的三大特点

- 确定性：如果输入相同 → 计算出来的哈希也相同

确定性

- 雪崩效应：即使输入变化很微小 → 哈希值剧烈变化（“hello” vs “hallo”）
- 平均分布：避免碰撞，让不同输入尽可能散列到不同位置

平均分布

正因为这些特点，传统哈希常用在：密码存储（确定性）、文件校验（雪崩效应检测篡改）、快速查找（平均分布的哈希表）、去重（确定性判断相同）等场景。

但这对向量检索没用，哈希的特点恰恰是向量检索最不想要的：

> 向量1: [0.8, 0.9, 0.1] → 哈希 → "a3f8d9e2" 向量2: [0.75, 0.85, 0.15] → 哈希 → “b7c4e1f5” # 虽然很相似，但哈希完全不同 向量3: [0.1, 0.2, 0.9] → 哈希 → “c9a2b6d3” 结果：无法通过哈希值判断向量1和向量2相似

LSH的创新：反其道而行之

LSH（Locality-Sensitive Hashing）的核心思想：相似的向量应该有更高概率被映射到相同的哈希值。

这恰好与传统哈希相反：

- 传统哈希：微小差异 → 完全不同的哈希值（避免碰撞）
- LSH哈希：相似输入 → 相同或相近的哈希值（主动让相似向量“碰撞”）

工作原理

LSH 使用**超平面（分割线）**来切分空间，相似的向量因为离得近，所以大概率在同一侧。

> 关于超平面： 简单理解，超平面就是高维空间中的"分界线"。2D平面可以用一条直线分割，3D空间可以用是一个2D平面来分割。不需要深究数学定义，只需要知道它的作用是"把空间一分为二"就行。 尤其不要去想象高维度的向量（1535维）到底是怎么被划分的。人类是3维生物，很难想象高维度的世界，这些只存储于数学中，而非能想象出来。

具体算法示例

假设有三个向量：

- 向量1: [0.8, 0.9]
- 向量2: [0.75, 0.85] ← 与向量1相似
- 向量3: [0.1, 0.2] ← 与向量1很不同

步骤1：随机生成一条分割线 比如：x + y = 1

步骤1：随机生成一条分割线

```
x + y = 1
```



步骤2：判断每个向量在线的哪一侧

用分割线 x + y = 1 来判断。

判断规则：

- 如果 x + y > 1，向量在分割线右侧

```
x + y > 1
```

- 如果 x + y < 1，向量在分割线左侧

```
x + y < 1
```

可以看到：

> 线左侧(x+y<1): B、D； 线右侧(x+y>1): A、C;

步骤3：相同哈希码的分到同一桶

- 点A、点C → 哈希码都是 “1” → 落入 bucket_1 ← 相似，同桶
- 点B、点D → 哈希码都是 “0” → 落入 bucket_0 ← 不同，不同桶

查询时

```
查询向量: [0.78, 0.88]
 ↓ 计算: 0.78 + 0.88 = 1.66 > 1
 ↓ 落入 bucket_1
 ↓ 只在 bucket_1 中搜索
 找到向量1、向量2（跳过向量3）
```

选择建议

- 首选HNSW：大多数场景下性能最优，闭着眼睛选它就对了

首选HNSW

- 成本敏感选IVF：内存有限或数据量特别大

成本敏感选IVF

- LSH：仅在超高维（>5000维）或特殊需求时考虑

6.2 主流向量数据库对比

聊完了向量数据库的几种搜索模式，看看市面上常见的向量数据库。

Chroma（最适合入门）

- 开源免费，不花一分钱
- 支持内存模式和持久化
- 适合中小规模数据（<100万）

索引算法

- 默认使用 HNSW（就是前面讲的第一种算法）

HNSW

- 不支持切换其他算法（IVF、LSH）

Milvus（企业级首选）

- 开源，CNCF孵化项目（跟大名鼎鼎的Kubernetes是一家人）
- 支持亿级数据，性能强悍
- 分布式架构，高可用

- 支持多种算法：HNSW、IVF、DiskANN 等（前两种就是上面讲的）

HNSW、IVF、DiskANN

- 可根据场景灵活选择： 小数据量选 HNSW（速度快） 大数据量选 IVF（省内存） 超大数据量选 DiskANN（用磁盘）
- 这就是 Milvus 的核心优势：不同场景用不同算法

6.3 实战：Chroma快速上手

下面用Chroma来实现一个完整的向量检索流程。

环境准备

在开始之前，需要安装必要的Python库：

```
# 安装Chroma向量数据库
pip install chromadb

# 安装通义千问SDK（用于生成向量）
pip install dashscope
```

依赖说明

- chromadb：轻量级向量数据库，支持本地存储和查询
- dashscope：阿里云通义千问SDK，提供Embedding API

核心代码示例

> [!TIP] 完整源码参考：samples/chapter5/chroma_faq_search.py

下面展示关键步骤的核心伪代码：

```
# 步骤1：创建客户端和集合
client = chromadb.PersistentClient(path="./chroma_db")
collection = client.create_collection(
    name="company_faq",
    metadata={"hnsw:space": "cosine"}  # 文本检索必须指定！
)

# 步骤2：生成向量并存储
documents = ["如何申请年假？ 员工入职满一年后...", ...]  # FAQ文本
embeddings = embedding_api.call(documents)  # 调用API生成向量
collection.add(ids=ids, embeddings=embeddings, metadatas=metadatas)

# 步骤3：查询检索
query_embedding = embedding_api.call("年假怎么申请？")  # 查询向量化
results = collection.query(query_embeddings=[query_embedding], n_results=3)

# 步骤4：处理结果
for metadata, distance in zip(results['metadatas'][0], results['distances'][0]):
    similarity = 1 - distance  # 转为相似度
    print(f"相似度：{similarity:.4f}")
    print(f"答案：{metadata['answer']}")
```

运行输出示例

```
正在生成向量并存储...
 成功存储 5 条FAQ

 查询: 年假怎么申请?
 找到 3 条相关FAQ:
#1 [相似度: 0.7640] [人事制度]
   问题: 如何申请年假?
   答案: 员工入职满一年后，可在OA系统提交年假申请，需提前3天申请，由直属领导审批。
#2 [相似度: 0.5762] [人事制度] ...
#3 [相似度: 0.5065] [人事制度] ...

 查询: 请假需要准备什么材料?
 找到 3 条相关FAQ:

#1 [相似度: 0.5762] [人事制度]
   问题: 病假需要什么证明?
   答案: 病假需提供医院诊断证明，3天以上需提交病假条，由HR审批。
#2 [相似度: 0.4790] [人事制度] ...
#3 [相似度: 0.4056] [人事制度] ...
```

6.4 Chroma 的两层结构

在深入代码细节之前，先了解 Chroma 的核心结构。

Chroma 采用简洁的两层设计：

```
客户端 (Client)
  |
  +- 集合1(Collection): company_faq
  +- 集合2(Collection): product_docs
  +- 集合3(Collection): ...
```

- 客户端（Client）：数据库的入口，负责管理所有数据 持久化模式：PersistentClient(path="./chroma_db") — 数据保存到磁盘 内存模式：Client() — 数据只存在内存，重启后消失
- 集合（Collection）：实际存储向量的容器，类似传统数据库的“表” 每个集合有独立的名称（如 company_faq） 每个集合可配置不同的相似度算法 不同业务场景建议使用不同集合

就像在 MySQL 中先连接数据库，然后操作具体的表一样：Chroma 先创建客户端，再在客户端下创建多个集合。

6.5 相似度算法选择

```
# 创建集合时指定使用余弦相似度
collection = client.create_collection(
    name="company_faq",
    metadata={
        "description": "公司FAQ知识库",
        "hnsw:space": "cosine"  # 使用余弦相似度
    }
)
```

为什么创建集合时要指定相似度算法?

向量数据库不只是存储向量，更重要的是要考虑后续的查询。而查询非常依赖索引，不同的相似度算法会导致完全不同的索引组织方式：

- 余弦相似度：关注向量方向，索引会把方向相近的向量聚在一起（适合文本）
- 欧氏距离：关注空间位置，索引会把空间距离近的向量聚在一起（适合图像、坐标）

欧氏距离

这就像建图书馆：

- 按主题分类（类似余弦）：把“编程”“历史”“小说”分开放

主题

- 按书的厚度分类（类似欧氏）：把薄书、中等厚度、厚书分开放

书的厚度

如果不设置会怎样

- Chroma默认使用L2距离（欧氏距离），不适合文本检索
- 会导致相似度分数偏低，甚至出现负数
- 检索结果非常离谱，几乎都是不相关内容

一旦建好索引，就无法更改算法了。所以文本检索必须在创建集合时显式指定 "hnsw:space": "cosine"

Chroma相似度计算方式对比

计算方式	参数值	适用场景	相似度范围	说明
余弦相似度（推荐）	"cosine"	文本、语义	[0, 1]	1表示完全相同，0表示无关
欧氏距离	"l2"	图像、坐标	数值复杂	不推荐用于文本
内积	"ip"	归一化向量	数值复杂	特殊场景使用

6.6 持久化模式

```
# 持久化：数据保存到磁盘，重启不丢失
# 代码执行后会在当前Python文件的同级创建一个名为chroma_db的目录，并存储数据库文件
client = chromadb.PersistentClient(path="./chroma_db")

# 内存：数据只在内存，重启后丢失（适合测试）
client = chromadb.Client()
```

6.7 存储参数详解

以下4个参数是Chroma定义的标准参数，不是随意定义的。

```
collection.add(
    ids=ids,              # 唯一标识符
    embeddings=embeddings,  # 向量（用于检索）
    documents=documents,    # 原始文本（用于调试）
    metadatas=metadatas     # 业务数据（用于展示）
)
```

各个参数的作用

参数	必须	作用	举例
ids	是	唯一标识，用于更新/删除	"faq_001"
embeddings	是	向量数据，用于相似度检索	[0.123, -0.456, ...]
documents	可选	原始文本，即向量数据对应的文本	"如何申请年假？..."
metadatas	可选	业务数据，返回给用户展示	{"question": "...", "answer": "..."}

为什么需要 documents

虽然检索只用 embeddings，但存储 documents 有两个好处：

- 调试方便：查看检索结果时，能看到原始文本，验证检索是否准确

调试方便

- 数据完整：不需要另外维护一个数据库来存储原文

数据完整

为什么需要 metadatas

metadatas 是返回给用户的关键信息：

```
# 检索后返回的是 metadata，不是embedding
for metadata in results['metadatas'][0]:
    print(f"问题: {metadata['question']}")
    print(f"答案: {metadata['answer']}")
    print(f"类别: {metadata['category']}")
```

如果不存 metadatas，检索结果只有 ID，还需要去其他数据库查询详情，非常麻烦。

最佳实践

- ids: 必须唯一，建议用有意义的编号（如 faq_001）

```
faq_001
```

- embeddings: 必须与API返回的向量一致
- documents: 建议存储，方便调试
- metadatas: 存储所有需要展示的字段（问题、答案、类别等）

6.8 相似度转换

其实代码：

```
results = collection.query(
        query_embeddings=[query_embedding],
        n_results=top_k
    )
```

已经查询出了最相似的k条结果，results就是这个结果。

后续的代码在做什么？ Chroma确实查询出了结果，但它的结果指标并不是相似度，而是distance（距离）。

```
# Chroma返回的是distance（余弦距离）
results = collection.query(
    query_embeddings=[query_embedding],
    n_results=3
)

# results['distances'][0] = [0.24, 0.42, 0.49]  # 这是距离值，不是相似度！

# 需要转换成相似度才好理解
for distance in results['distances'][0]:
    similarity = 1 - distance  # 转换公式
    print(f"相似度: {similarity:.4f}")  # 0.76, 0.58, 0.51
```

6.9 扩展阅读：为什么 similarity = 1 - distance

这是余弦距离的数学定义：余弦距离 = 1 - 余弦相似度。

余弦距离 = 1 - 余弦相似度

余弦相似度 = cosθ（θ 是向量夹角），范围 [-1, 1]。为了获得一个"距离"指标，数学上直接用 1 - cosθ 来定义余弦距离。三个边界值对比如下：

场景	cosθ	余弦相似度	余弦距离
完全相同	1	1	0
完全相反	-1	-1	2
正交（无关）	0	0	1

可以看到：余弦相似度越高（接近 1），余弦距离越小（接近 0）。数值一一对应，用 相似度 = 1 - 距离 就能互相转换。

Chroma 配置了 "hnsw:space": "cosine" 后，内部计算的就是余弦距离。所以从 results 拿到 distance 后，用 1 - distance 就能还原回余弦相似度。

[!TIP] 下一节预告 现在已经掌握了向量生成、向量检索的完整流程。但在实际项目中，还有关键问题：如何把文档切分成合适的片段？ 一份完整的技术手册可能有几十页，是整个文档生成一个向量？还是按章节切分？按段落切分？切得太大会导致检索不精准，切得太小又会丢失上下文。 下一节《第一个RAG应用：游戏知识问答》，实战一个完整的RAG应用，从文档处理、向量化、检索到生成，把前面学到的所有知识串起来。

6.10 ■ 学点英语

中文	English	音标	说明
局部敏感哈希	Locality-Sensitive Hashing (LSH)	/loʊˈkæləti ˈsensətɪv ˈhæʃɪŋ/	将相似向量映射到同一哈希桶的近似搜索算法
HNSW 图	HNSW Graph	/eɪtʃ en es ˈdʌbəl juː græf/	分层小世界图，通过多层跳表结构实现对数级搜索复杂度
Chroma	Chroma	/ˈkroʊmə/	轻量级开源向量数据库，适合学习和原型开发
余弦距离	Cosine Distance	/ˈkoʊsaɪn ˈdɪstəns/	1 - 余弦相似度，Chroma中配置cosine后返回的距离值
向量集合	Collection	/kəˈlekʃn/	Chroma中组织和存储向量的基本单位